



Spring MVC with database

1. Spring MVC with JDBC
2. JdbcTemplate
3. Example using JDBC
4. Spring MVC with ORM
5. HibernateTemplate
6. Example using ORM

- Spring provides a simplification in handling database access with the Spring JDBC Template class.
- Advantages of JdbcTemplate:
 - Avoiding writing all JDBC steps explicitly
 - Easy exception handling
 - No need of closing multiple resources
 - Easy to use methods for querying the database
 - Simplified transaction management

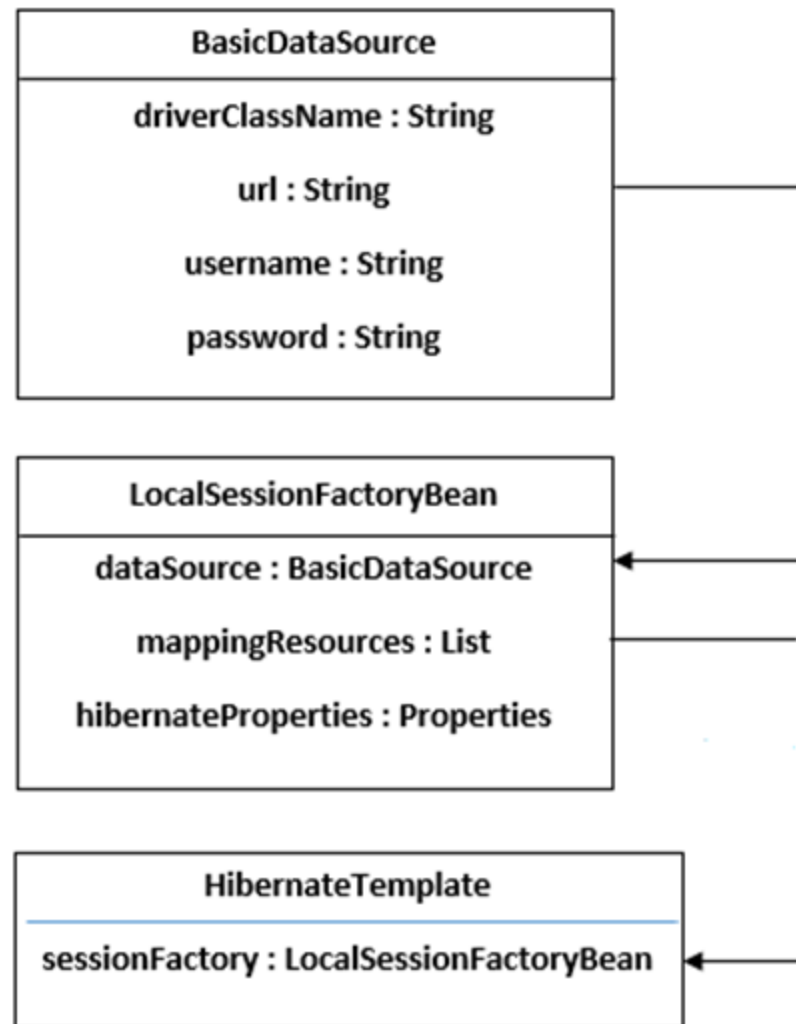
- It takes care of creation and release of resources
- We can perform all the database operations by the help of JdbcTemplate class such as insertion, updation, deletion and retrieval of the data from the database.
- Usage
 - Step 1 – Create a JdbcTemplate object using a configured datasource. This instance is required generally from DAO layer. Datasource can be configured in bean definition file
 - Step 2 – Use JdbcTemplate object methods to make database operations. Major methods are query() for select operations and update() for DML operations
- ResultSetExtractor and RowMapper interfaces are available for converting result set into object or list of objects.

- Add the maven dependency for adding spring-jdbc in pom.xml of maven project
- Configure JdbcTemplate as spring bean which needs datasource bean. The DriverManagerDataSource is used to contain the information about the database.
- Add the database operation methods in DAO class. DAO class can be used from controller class as per need of the request

- Spring provides a simplification in handling database access with the spring ORM module.
- Spring provides API to easily integrate Spring with ORM frameworks such as Hibernate.
- Advantage of ORM Frameworks with Spring
 - Less coding is required.
 - No need to write queries. All operations are in terms of objects.
 - Easy to test
 - Better exception handling
 - Integrated transaction management.

5. HibernateTemplate

- HibernateTemplate class eliminates steps like create Configuration, BuildSessionFactory, Session, beginning and committing transaction etc.
- It offers a variety of methods for performing CRUD (Create, Read, Update, Delete) operations, executing queries, and managing transactions.
- Dependency for creating hibernate template instance



- Add the maven dependency for adding spring orm, hibernate core and mysql connector in pom.xml.
 - Configure HibernateTemplate as spring bean.
 - Add the database operation methods in DAO class
 - Use DAO class from service or controller layer
 - Manage transactions by declaring bean for HibernateTransactionManager. This bean works in co-ordination with @Transactional annotation added in the service layer either above the class or above the method.
- HibernateTransactionManager bean should be set as transaction-manager for annotation-driven transaction manager